

SDM Complaint Tracker — Source of Truth

Oct 3, 2026 · @Aditya

Status and how to use this doc

This is the single reference for the SDM office complaint tracker: problem, decisions, design, build plan and coding prompts. Everything here is decided **except deployment**, which is still open (see Deployment strategy).

Area	Status
Problem statement	Final
Architecture and tech stack	Decided: Django + Postgres + HTMX, one server
Data model and state machine	Decided
v1 scope	Decided: tracking only, no notifications
Coding	Starting now
Deployment	Not fixed. Oracle free tier tried, blocked by capacity. Options listed below.
Office data (departments, villages, SLAs, users)	Not yet collected

How to use it:

- Put a copy of this doc in the code repo as `docs/PROJECT_SPEC.md` so Copilot can read it.
- When a decision changes, update the relevant section and add a line to the Decision log at the end.
- If code and this doc disagree, fix one of them the same day.

Problem statement

The SDM office (Mandi Dabwali, Haryana) cannot reliably track where each citizen complaint is, because one complaint goes back and forth between offices many times before it closes.

Who and where

- **Office:** Sub-Divisional Magistrate (SDM) office. The officer is referred to as "the DM" in our discussions.
- **Users:** the dealing clerk / reader who registers complaints, branch and department officers, and the SDM who reviews and approves.
- **Sources of complaints:** CM Window (Haryana's CM grievance portal), Jan Samvad, Samadhan Shivir, social media, and walk-in / internal petitions.
- **Departments a complaint can be marked to:** Tehsildar, BDPO, DRO, Revenue Branch, Accounts Branch, Establishment Branch, Grievance Cell, Police Station, Municipal Council (list to be confirmed by the office).

The to-and-fro loop (the core problem)

1. A complaint is received and registered.
2. The office sends a **notice** to the concerned department to inquire.
3. The department sends back an **inquiry report**.
4. The SDM reviews the report.
 - Not satisfied: a fresh notice goes back to the department (round 2, 3, ...).
 - Satisfied: the report / ATR (Action Taken Report) is uploaded to the **CM portal**.
5. The CM portal can **reject** the upload. The office then has to fix it locally or start a fresh inquiry.
6. Only when the portal accepts is the complaint closed.

So one complaint can have several notices, several inquiry reports and several portal attempts. Each step produces a document.

Pain points today

- No single view of where a complaint is right now and for how many days.
- Departments sit on complaints with no ageing alert; delays surface only at district reviews.

- Notices, inquiry reports and ATRs live in separate paper bundles and Drive folders, not linked to the complaint.
- Shared spreadsheets break: formulas get overwritten, rows get reordered, no history of who changed what.
- The same citizen often files the same grievance on several platforms, and nobody links them.

What the DM wants

- A dashboard that anyone with access can use.
- Register a complaint from its document (one page, computer typed) and store its details in a database.
- Tag and filter every complaint: complaint number, complainant phone, village, department, platform, status, age.
- Attach every document of the to-and-fro to the complaint, and see where it went (department, CM portal) and when.
- Click a complaint and see everything: details, all documents, full movement timeline and a short summary.

Success looks like

- Any complaint is found in seconds by number, phone, name, village or text.
- "Where is it now, since when, and what happened before" is answered on one screen.
- Overdue complaints are visible every morning without anyone compiling a list.
- Data entry per complaint takes under 60 seconds, or the office will not use it.

What the DM already built, and why we replace it

The DM's prototype is a good **process and UI spec** and a weak **storage design**. We keep its screens and fields; we replace Google Sheets and Drive with a real database.

File the DM shared	What it is	What we keep
Complaint_Tracker_Prototype_v2.xlsx	Google Sheet with tabs: Dashboard, Complaints (Master), Movement Log, Office & Branch List, Read Me	The fields (complaint no., date received, village, platform, complainant, subject, description, marked to, dates, status, remarks) and the idea of a movement log
SDM_Complaint_App-3.html	Single-page app reading the Sheet every 30 s: 60/40 split view, search, filters (department, village, platform, status), ageing chips (>3, >7, >10 days, safe, resolved), case detail with documents and timeline, "New Intake" form	The whole UI layout and filters. Port it to Django templates + HTMX.
daily_reminder_script.gs	Apps Script that emails overdue complaints daily (orange at 3 days, red at 6)	The ageing thresholds. Email is on hold for v1.
SDM_Complaint_Tracker_Documentation (two .docx)	Problem write-up, architecture (Sheets + Drive + AppSheet), column spec, formula logic	The problem framing and the three document stages (complaint, department report, ATR)

Why Sheets + Drive does not hold up

- **One row per complaint with fixed columns** ("Department Report PDF", "ATR PDF") cannot hold round 2, round 3, or a second portal attempt. The loop gets overwritten.
- No constraints: anyone can type anything in any cell, and formulas get overwritten.
- No transactions: two clerks editing at once lose updates.
- No real access control: every editor sees every complainant's phone and address.
- No audit trail: "who closed this case?" cannot be answered.
- Drive links break when sharing settings change; rows can point at deleted files.
- The 30-second polling of the whole sheet gets slow as rows grow.

The fix is not "a bigger sheet". It is modelling the loop as **rows, not columns**: each notice, each report and each portal attempt is its own record, linked to the complaint.

Scale and sizing

Scale is tiny: about 100 complaints a week (5,200–6,000 a year), so one small server runs this for ten years. Scale is not a design driver; correctness, search and audit are.

Inputs (from the office): up to 100 registered complaints a week (more arrive but many are rejected on the spot); each complaint is **one page, computer typed**.

Item	Per year	After 10 years
Complaints	~6,000	~60,000
Referrals (notices), ~1.7 rounds per case	~10,000	~100,000
Portal submissions	~7,500	~75,000
Documents (~6 per case)	~36,000	~360,000
Events (timeline rows, ~20 per case)	~120,000	~1.2 million
Pages stored (~8–10 per case)	50,000–60,000	~600,000
File storage (100–150 KB per page)	6–9 GB	60–90 GB
Database size (rows only)	small	2–3 GB

- **Write rate:** 200–300 writes a day, mostly in office hours. Postgres handles thousands per second.

- **Users at once:** about 10–20.
- **Server needed:** 1–2 vCPU, 1–6 GB RAM (1 GB works with a 2 GB swap file), 50 GB disk.
- **OCR load:** almost none. Typed complaints that arrive as digital PDFs need only `pdftotext`. Scanned typed pages: Tesseract, about 1 second a page.

The one scale question that matters: **will this ever run for more than one office?** To keep that door open for free, every main table carries an `office_id` from day one.

Guiding principles

Keep everything that records the to-and-fro; cut everything that is infrastructure around it. The goal is a tracker that looks and behaves like a real product, not a pile of services.

Core idea: append-only event log + derived current state

- Nothing is overwritten. Every action (notice issued, report received, review, portal upload, rejection) **adds** a row.
- The complaint's current status, current department and days pending are **computed** from those rows and cached on the complaint in the same transaction.
- This gives the timeline, the audit trail ("who did what, when") and pendency for free.

Cut from the design (not needed for v1)

Cut	Use instead	Why it is safe
Redis + Celery workers	A cron job / Django management command	Only one background task exists (daily overdue check)
S3 / MinIO object storage	A folder on the server's disk	6–9 GB a year
React / Vite frontend	Django templates + HTMX	No build step, works on old office browsers
Separate REST API + frontend	One Django app serving HTML	Add a JSON API later only if a mobile app is needed
Managed database (RDS etc.)	Postgres on the same server	200 writes a day

Cut	Use instead	Why it is safe
Meilisearch / Typesense	Postgres full-text search + trigram	Whole DB fits in RAM
Email / SMS / WhatsApp / Telegram notifications	An in-app "overdue today" page	On hold by decision; revisit after the office uses it
LLM / vision OCR	pdf totext ; Tesseract only for scans	Complaints are typed, one page
Kubernetes, microservices, staging env, Sentry	One server, log files	One office

Must not cut (these keep it from becoming the spreadsheet again)

1. **Referrals and portal submissions as separate rows** with round / attempt numbers.
2. **The append-only `event` table.**
3. **One function that performs every status change** (`apply_transition`). No view updates status directly.
4. **Database constraints** (foreign keys, unique round numbers, only one open referral per complaint).
5. **Migrations from the first commit.**
6. **Nightly backup with one tested restore** (once deployed).
7. **`office_id` on main tables** (one column, costs nothing now).

Product-quality checklist (what makes it not look vibe-coded)

- Departments, villages, categories and SLA days are **data**, editable in an admin screen, never hard-coded.
- Documents are never overwritten or hard-deleted; a correction is a new file that supersedes the old one.
- Every list screen uses one shared filter/search/pagination helper.
- Every case has a visible history tab built from the event log.
- Tests cover the state machine (every legal and illegal transition), pendency maths and permissions.
- Seed data + one-command local setup, so a demo works on any laptop.
- `DEBUG=False` in production, secrets only in `.env` , never in git.

Architecture and tech stack

One Django app, one Postgres database, files on local disk, one cron job, all on a single server behind Caddy. Nothing else.

Layer	Choice	Why
Language	Python 3.12	Your primary stack
Web framework	Django 5.x	Gives login, users, groups/permissions, admin panel, ORM, migrations, forms, file uploads, CSRF for free. Chosen over FastAPI because it removes weeks of plumbing.
UI	Django templates + HTMX (one script tag) + a light CSS (Pico.css or Tailwind via CDN)	Server-rendered; HTMX swaps the right-hand detail pane without a page reload, matching the DM's 60/40 prototype. No build step.
Admin / master data	Django admin	Departments, villages, categories, SLA days, users get CRUD screens with zero code
Database	PostgreSQL 16	Relational data, FKs and constraints, full-text search, <code>pg_trgm</code> for fuzzy names. Chosen over SQLite for better text search (Hindi + English) and safe concurrent writes.
File storage	Local folder (<code>MEDIA_ROOT</code>) on the server	Small volume. Files named by SHA-256 hash, so duplicates are stored once.
PDF text	<code>pdftotext</code> (poppler)	Typed PDFs give perfect text instantly
OCR (only for scanned pages)	Tesseract with <code>hin+eng</code>	Free, ~1 s a page, accurate on typed text
PDF viewing	Browser's built-in viewer or PDF.js	Free

Layer	Choice	Why
Background job	Django management command run by cron	Only for the daily overdue/pendency snapshot, if needed
Web server / HTTPS	Caddy	Automatic Let's Encrypt certificates; 3-line config
App server	Gunicorn	Standard for Django
Packaging	Docker Compose: db (Postgres), web (Django + Gunicorn), caddy	Same setup on a laptop and on any server; easy to move hosts later
Tests	pytest + pytest-django	
Code quality	ruff (lint + format)	

Request flow

1. Browser opens `https://<domain>/` → Caddy (HTTPS) → Gunicorn → Django.
2. Django reads/writes Postgres and saves uploaded files to the media folder.
3. Document downloads go through a Django view that checks the user's permission before streaming the file (files are never served from a public URL).
4. Every status change goes through `apply_transition()`, which writes the event, updates the derived fields and opens/closes referral or portal rows in one transaction.

Repository layout (suggested)

```

complaint-tracker/
  docs/PROJECT_SPEC.md          <- copy of this doc
  docker-compose.yml
  Caddyfile
  .env.example
  manage.py
  config/                       <- settings, urls, wsgi
  core/                         <- Office, Department, Village, Category,
User/roles
  complaints/
    models.py                   <- Complaint, Complainant, Referral,
InquiryReport,
                                PortalSubmission, Document, Event, Tag
    workflow.py                 <- states, transitions, apply_transition()
    services/                   <- intake, search, pendency, summary, documents
    views.py, urls.py, forms.py
    templates/complaints/
    tests/
  scripts/                      <- backup.sh, restore.sh (later)

```

Data model

Thirteen tables: five of master data, eight for the case and its loop. The loop lives in `Referral`, `InquiryReport` and `PortalSubmission`, one row per round or attempt. All tables except `Office` carry `office_id`. All carry `created_at`, `updated_at` unless noted.

Master data (edited in Django admin)

Table	Fields	Notes
<code>Office</code>	name, code, district, state	One row for now (SDM Mandi Dabwali)
<code>Department</code>	office, name, short_name, kind (<code>internal_branch</code> / <code>external_department</code>), head_name, email, phone, default_sla_days, is_active	Replaces the "Office & Branch List" tab

Table	Fields	Notes
Village	office, name, name_hi (Hindi), block, tehsil, is_active	Unique (office, name)
Category	office, name, sla_days (nullable, overrides department)	Land, pension, civic, police, etc.
User	Django user + office, department (nullable), phone, role via Django groups	Roles in the Access section

The case

Table	Fields	Notes
Complainant	office, name, phone (indexed), address, village	Phone is how repeat filers are found
Complaint	office, ref_no (unique per office, e.g. C-2026-0001), source_platform (CM Window / Jan Samvad / Samadhan Shivir / Social Media / Internal), external_ref (CM Window grievance ID, nullable), received_on, complainant, village, category, subject, description, tags (M2M), related_to (self FK, for duplicates), current_state , current_department , current_stage_started_at , closed_on, created_by	Bold fields are a derived cache, written only by <code>apply_transition()</code>
Referral	office, complaint, round_no , to_department, issued_on, due_on, notice_document (FK Document, nullable), closed_on (nullable), outcome (<code>report_received</code> / <code>no_response</code> / <code>superseded</code>), remarks, created_by	One row per notice sent to a department

Table	Fields	Notes
<code>InquiryReport</code>	office, referral, received_on, document, review_outcome (pending / satisfied / not_satisfied), review_note, reviewed_by, reviewed_on	The department's reply to one referral, and the SDM's verdict on it
<code>PortalSubmission</code>	office, complaint, attempt_no , submitted_on, document (ATR copy), portal_ack_no, outcome (pending / accepted / rejected), outcome_on, rejection_reason, created_by	One row per upload to the CM portal
<code>Document</code>	office, complaint, kind (complaint / notice / inquiry_report / atr / portal_ack / portal_rejection / other), file, sha256, original_filename, mime_type, size_bytes, page_count, extracted_text, text_status (none / done / failed), supersedes (self FK, nullable), uploaded_by, uploaded_at, is_deleted	Immutable. Never overwritten. Soft delete only.
<code>Event</code>	office, complaint, type, actor, occurred_at, payload (JSON)	Append-only. No update or delete in code. This is the timeline.
<code>Tag</code>	office, name	Free tags for filtering

Event types: `COMPLAINT_REGISTERED`, `NOTICE_ISSUED`, `REPORT_RECEIVED`, `REVIEW_SATISFIED`, `REVIEW_NOT_SATISFIED`, `PORTAL_SUBMITTED`, `PORTAL_ACCEPTED`, `PORTAL_REJECTED`, `CLOSED`, `REOPENED`, `DOCUMENT_ATTACHED`, `DETAILS_EDITED`, `NOTE_ADDED`.

Constraints (in the database, not only in Python)

- `unique (complaint, round_no)` on `Referral`; `unique (complaint, attempt_no)` on `PortalSubmission`.

- Partial unique index: only **one open referral** per complaint (`unique (complaint) where closed_on is null`).
- Partial unique index: only **one pending portal submission** per complaint.
- `unique (office, ref_no)` on `Complaint` .
- `unique (complaint, sha256)` on `Document` , so the same file is not attached twice to one case.
- All foreign keys `on_delete=PROTECT` ; nothing cascades away history.

Indexes

- `Complaint` : (office, current_state), (office, current_department), (office, received_on), village, source_platform.
- `Complainant.phone` .
- Full-text: a `SearchVector` (GIN index) over ref_no, subject, description, complainant name, village name, external_ref.
- `pg_trgm` GIN index on complainant name and subject for fuzzy matches ("Sunita" / "Suneeta").
- Full-text GIN index on `Document.extracted_text` .

v1 scope

v1 is a complaint tracker only: register, attach, move through the loop, search, see everything on one screen. Six screens, five of them custom.

Screens

#	Screen	What it does
1	Login	Django auth, username + password

#	Screen	What it does
2	Complaint list (left 60%)	Search box; filters for department, village, platform, status, category, date range, tag; ageing chips (All, >10 days critical, >7 overdue, >3 warning, 0-2 safe, Closed); KPI strip at top (total, pending, each ageing bucket, closed); cards showing ref no, village, platform, subject, complainant, current department, "Pending at X for N days"
3	Complaint detail (right 40%, loaded by HTMX; full page on mobile)	Header (ref no, subject, complainant, phone, village, platform, external ref); status box with summary sentence; description; documents list grouped by kind with view/download; timeline from the event log; action buttons valid for the current state only
4	New complaint (intake)	Upload the complaint PDF first; fields pre-filled from extracted text; clerk corrects and saves; duplicate warning by phone / external ref / similar subject
5	Overdue today	All open cases past their SLA, grouped by department, oldest first; portal rejections waiting for action; printable for review meetings
6	Django admin	Departments, villages, categories, SLA days, users, roles

Actions on a complaint (each one = one `apply_transition` call + optional document upload)

- Issue notice to department (choose department, due date auto-set from SLA, attach notice)
- Record inquiry report received (attach report)
- Mark report satisfied / not satisfied (with a note)
- Record portal submission (attach ATR, portal ack no.)

- Record portal accepted / rejected (rejection reason; choose: fix locally or send new notice)
- Reopen a closed case (reason required)
- Attach other document; add note; edit details (logged as `DETAILS_EDITED` with old and new values)

Intake rules

- **Digital PDF:** extract text with `pdftotext`, try to read complaint no., name, phone, village, subject with simple patterns. Pre-fill; the clerk confirms.
- **Scanned page / photo:** Tesseract (`hin+eng`), same pre-fill.
- **Nothing is saved until the clerk clicks Save.** Extraction only suggests.
- Duplicate check before save: same external ref, same phone, or very similar subject in the same village → show "possibly related to C-2026-0045" with a link and an option to mark as related.
- Target: under 60 seconds per complaint.

Search

- One box searches ref no, external ref, complainant name, phone, subject, description, village, and extracted document text.
- Postgres full-text search plus trigram for spelling variants. Store Hindi as typed; search both scripts where a transliterated name exists.
- Filters combine with search; results paginated (25 per page).

Pendency and ageing

- **Days at current office** = today - `current_stage_started_at` (date difference, calendar days).
- **Total age** = today (or `closed_on`) - `received_on`.
- **SLA days** = `category.sla_days` if set, else `department.default_sla_days`.
- Ageing buckets match the DM's prototype: 0-2 safe, 3+ warning, 7+ overdue, 10+ critical. Keep thresholds in settings, not in templates.

Summary sentence

Built from the event log by plain code (no LLM), for example: "Received 13 Sep from CM Window. Notice to Tehsildar 15 Sep. Report received 22 Sep, not satisfied. Second notice 24 Sep, pending 6 days."

Out of scope for v1

- Email, SMS, WhatsApp, Telegram reminders (on hold)
- Any integration with the CM Window portal (no public API; the office uploads there by hand)
- Mobile app, public citizen-facing status page, LLM summaries, multi-office rollout, reports beyond the overdue page

Roles, access and security

Five roles, each a Django group; department officers see only their own department's cases. Every change records who made it.

Role	Can do	Sees
Clerk	Register complaints, attach documents, issue notices, record reports received, record portal submissions and outcomes	All cases in the office
Branch / department officer	Attach documents and notes on cases currently with their department	Only their department's cases
Reviewer (SDM)	Everything a clerk can, plus mark reports satisfied / not satisfied, close and reopen	All cases
Admin	Django admin: master data, users, roles	All
Auditor	Read only	All, including history

Rules

- Permission checks live in one place (a queryset helper `visible_complaints(user)` and a function `can(user, action, complaint)`), not scattered in views.
- No shared logins. Every user is a named person, so the event log means something.
- Department scoping is enforced in the query, not only hidden in the UI.
- Expect low login discipline: design for one or two power users doing data entry and everyone else reading.

Privacy (complainant data is personal data under India's DPDP Act)

- Phone numbers and addresses visible only to logged-in users with a role; never in URLs or logs.
- Documents served only through a permission-checked view, never from a public `/media/` URL.
- No real complaint data on personal laptops or in git. Use seed / fake data for development.
- Decide a retention period with the DM (open question).

Security checklist before real data goes in

- ☐ `DEBUG=False`, `ALLOWED_HOSTS` set to the real domain
- ☐ `SECRET_KEY`, DB password and any keys only in `.env`; `.env` is in `.gitignore`
- ☐ HTTPS only; secure and HttpOnly session and CSRF cookies
- ☐ Postgres not reachable from the internet (only ports 22, 80, 443 open)
- ☐ Upload limits: PDF / JPG / PNG only, max ~10 MB per file
- ☐ Strong admin password; admin URL not at the default `/admin/`
- ☐ Backup and restore tested once by hand

How to proceed

Build the core first and prove the loop works before any screen. Each step ends with something you can run.

1. **Project skeleton.** Django project, Docker Compose (Postgres + web), `.env.example`, `ruff`, `pytest`, settings split (base / dev / prod). Done when `docker compose up` shows the Django welcome page and `pytest` runs.

2. **Models + migrations + admin.** All 13 tables, constraints and indexes from the Data model section. Register master data in admin. Seed command with 1 office, 9 departments, 7 villages, 5 categories, 5 users (one per role), 30 fake complaints spread across all states. Done when admin works and `python manage.py seed` fills it.
3. **State machine + tests (no UI).** `complaints/workflow.py` with `apply_transition()`. Table-driven tests for every legal and illegal transition, round/attempt numbering, the one-open-referral rule, and pendency maths. Done when tests pass and a script can push one complaint through 3 notice rounds, 2 portal rejections and closure.
4. **List + detail screens.** Port the DM's 60/40 layout to templates + HTMX. Search, filters, ageing chips, KPI strip, timeline, documents, summary sentence, action buttons that call `apply_transition`. Done when the whole loop can be clicked through.
5. **Intake.** Upload PDF → extract text → pre-filled form → duplicate check → save. Done when a real (anonymised) complaint PDF registers in under 60 seconds.
6. **Overdue today page + permissions.** Role groups, department scoping, auditor read-only. Done when a department officer sees only their cases.
7. **Polish.** Empty states, error pages, mobile layout, print style for the overdue page, admin URL change, security checklist.
8. **Deployment** (when the hosting decision is made): see Deployment strategy.
9. **Pilot with the office.** Load real master data, train the clerk, run for 2–4 weeks, collect feedback. Only then consider notifications.

Definition of done for any feature

- Status changes only through `apply_transition`.
- Has at least one test for its main rule.
- Works with the seed data.
- Writes an `Event` when it changes anything about a complaint.
- Respects `visible_complaints(user)`.

Deployment strategy (NOT FIXED)

The hosting choice is still open. What is fixed is the target shape: one Linux server running Docker Compose (Postgres + Django + Caddy), reached over HTTPS on a free domain. Any host that gives a Linux VM with SSH works; the app does not depend on the host.

Constraints for the pilot

- Free or near-free (pilot is personal, not yet funded by the office).
- India region preferred (complainant data).
- No notifications needed, so no email/SMS accounts.
- If the office adopts it officially, it will likely need to move to NIC / state data centre hosting, requested by the office itself. Plain Docker containers keep that move small.

Options

Option	Cost	Status / verdict
Oracle Cloud Always Free, VM.Standard.A1.Flex (ARM), Mumbai	₹0	Tried, blocked. "Out of capacity" in AD-1 at both 2 OCPU/12 GB and 1 OCPU/6 GB. Can retry at off-peak hours.
Oracle Cloud Always Free, VM.Standard.E2.1.Micro (AMD, 1 OCPU, 1 GB)	₹0	Not tried yet. Less contested. Needs a 2 GB swap file. Next thing to try.
Paid Indian VM (E2E Networks, DigitalOcean Bangalore)	roughly ₹500–1,800 / month	Reliable fallback; works the same day. E2E is MeitY-empanelled, useful for the government conversation.
Render / Railway (managed Django + Postgres)	free tier / small	Fits the app shape. Free web service sleeps when idle; check current free-tier limits before choosing.
NIC MeghRaj (cloud.gov.in)	allocated to govt	Not for the pilot. Govt-only onboarding through the department; days to weeks. Right home only if the office adopts it.
Vercel	free	Rejected. Serverless, no persistent process, no durable disk, no Postgres. Wrong shape for Django.

Option	Cost	Status / verdict
PythonAnywhere free	₹0	Demo only (MySQL on free tier, limited).
Google Sheets / Drive as backend	₹0	Rejected (see why we replace it).

Prices are approximate and change often; check before paying.

Oracle setup done so far (to resume from)

- Account created, home region **India West (Mumbai)**, Free Trial (drops to Always Free after 30 days; nothing is billed unless upgraded).
- VCN `sdm-tracker-vcn` created with **Networking → Virtual Cloud Networks → Start VCN Wizard → "Create VCN with Internet Connectivity"** (the inline "create new VCN" inside the instance wizard did not allow a public IP).
- SSH key generated on the laptop: `ssh-keygen -t ed25519 -C "sdm-tracker"`; public key = `~/.ssh/id_ed25519.pub` (one line starting `ssh-ed25519`).
- Instance settings that were accepted: Ubuntu 24.04, existing VCN, **public** subnet, public IPv4 on, SSH key pasted, boot volume 50 GB, default performance (VPU 10). The cost estimator shows only the boot volume at list price; it ignores the free tier.
- Instance creation failed with "Out of capacity".

Steps once a server exists (any host)

1. Note the public IP; SSH in: `ssh ubuntu@<ip>`.
2. Open ports 80 and 443 in the cloud firewall (Oracle: subnet → Security List → Ingress rules, TCP 80 and 443 from 0.0.0.0/0).
3. **Oracle only:** also open them in the server's own iptables, or traffic stays blocked:

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo netfilter-persistent save
```

4. Free domain: sign in at duckdns.org, create e.g. `sdm-dabwali.duckdns.org`, set it to the server IP.
5. On a 1 GB server, add a 2 GB swap file.

- 6. Install Docker, copy the repo, create `.env`, `docker compose up -d`, run migrations, create the admin user, load master data.
- 7. Caddyfile is three lines; Caddy fetches the HTTPS certificate itself:

```
sdm-dabwali.duckdns.org {  
    reverse_proxy web:8000  
}
```

- 8. Backups: nightly cron `pg_dump` + media folder, copied off the server (e.g. rclone to Google Drive or Cloudflare R2 free tier). Write `restore.sh` and test it once by hand.
- 9. Turn on `unattended-upgrades` for OS patches.
- 10. Verify from the laptop: `nc -zv <ip> 5432` must fail; `https://<domain>` shows a padlock.

Claude Code running **on your laptop** (it needs your SSH key) can do steps 3 and 5-10. Account sign-ups, payments, OTPs and DNS are manual.

Running cost once decided

Setup	Monthly
Oracle free + DuckDNS + free backup target	₹0
Paid Indian VM + DuckDNS	roughly ₹500-1,800
Later, if notifications are added: SMS (DLT)	+₹400-750

Copilot prompts

Use one master context file plus one prompt per build step. Give Copilot one step at a time, review, commit, then move on.

Setup

- 1. Save this doc into the repo as `docs/PROJECT_SPEC.md`.
- 2. Save the master prompt below as `.github/copilot-instructions.md`. Copilot reads that file automatically on every chat in the repo.
- 3. In Copilot Chat (agent mode), paste the step prompts in order.

Master context (save as `.github/copilot-instructions.md`)

Project: SDM Office Complaint Tracker

Full spec: docs/PROJECT_SPEC.md. Read it before large changes. If code and spec disagree, ask.

What this is

A Django app for a sub-divisional (SDM) office in Haryana to track citizen complaints

through a repeating loop: notice to a department -> inquiry report -> SDM review

(may loop back) -> upload to CM portal (may be rejected and loop back) -> closed.

Scale: ~100 complaints/week, one office, 10-20 users. Do not over-engineer.

Stack (do not add to it without asking)

Python 3.12, Django 5, PostgreSQL 16, Django templates + HTMX, Pico.css or Tailwind CDN,

Gunicorn, Caddy, Docker Compose, pytest + pytest-django, ruff.

NOT allowed: React/Vite, Celery/Redis, DRF API layer, S3, microservices, LLM calls.

Hard rules

1. Every status change of a Complaint goes through `complaints/workflow.py::apply_transition()`.

No view, form, admin action or signal may set `current_state`, `current_department` or `current_stage_started_at` directly.

2. `apply_transition()` runs in one DB transaction: validates the transition, writes one Event, creates/closes Referral / InquiryReport / PortalSubmission rows, updates the derived fields.

3. Event is append-only. Never update or delete Event rows.

4. Documents are immutable. Never overwrite a file. Corrections = new Document with `supersedes=old`.

Soft delete only (`is_deleted`). Store files under a path built from the SHA-256 hash.

5. Departments, villages, categories, SLA days, ageing thresholds are data or settings, never

hard-coded in templates or views.

6. Every main table has office (FK). Every query goes through `visible_complaints(user)`.

7. Constraints live in the database (unique round_no per complaint, unique attempt_no per complaint, one open Referral per complaint, one pending PortalSubmission per complaint, unique ref_no per office). Use on_delete=PROTECT.
8. Files are served only through a permission-checked view, never a public media URL.
9. Secrets only via environment variables (.env). DEBUG from env, default False.
10. Every new rule gets a test.

States

RECEIVED, AWAITING_REPORT, UNDER_REVIEW, APPROVED, PORTAL_SUBMITTED, CLOSED

Transitions (action: from -> to : side effects)

- issue_notice: RECEIVED -> AWAITING_REPORT : new Referral round 1, due_on from SLA
 - reissue_notice: AWAITING_REPORT -> AWAITING_REPORT : close open Referral (outcome no_response or superseded), new Referral round+1 (department may change)
 - record_report: AWAITING_REPORT -> UNDER_REVIEW : close Referral (report_received),
new InquiryReport (review_outcome pending) with its Document
 - review_not_satisfied: UNDER_REVIEW -> AWAITING_REPORT : report not_satisfied + note,
new Referral round+1
 - review_satisfied: UNDER_REVIEW -> APPROVED : report satisfied + note
 - submit_to_portal: APPROVED -> PORTAL_SUBMITTED : new PortalSubmission attempt+1 (pending)
 - portal_accepted: PORTAL_SUBMITTED -> CLOSED : submission accepted, closed_on set
 - portal_rejected: PORTAL_SUBMITTED -> APPROVED | UNDER_REVIEW | AWAITING_REPORT :
submission rejected + reason; next_step = resubmit | review_again | new_inquiry
(new_inquiry also creates a new Referral round+1)
 - close_without_portal: APPROVED -> CLOSED : reason required (sources that need no CM portal upload)
 - reopen: CLOSED -> UNDER_REVIEW : reason required
- Any other (state, action) pair raises InvalidTransition.

Derived fields

current_department = the open Referral's department in AWAITING_REPORT, else

```

null
(shown as "SDM office", or "CM portal" in PORTAL_SUBMITTED).
current_stage_started_at = time of the last transition.
Days at current office = today - current_stage_started_at (calendar days).
SLA days = category.sla_days or department.default_sla_days.
Ageing buckets (settings): 0-2 safe, 3+ warning, 7+ overdue, 10+ critical.

## Style
Small functions, type hints, services in complaints/services/, thin views.
Plain, readable code. No clever metaprogramming.

```

Step prompts (paste one at a time)

Step 1 — skeleton

```

Create the project skeleton described in docs/PROJECT_SPEC.md (Architecture
and tech stack,
Repository layout). Django 5 project named config, apps core and complaints.
Settings split
into base/dev/prod reading env vars (django-environ). docker-compose.yml with
services db
(postgres:16) and web (Django + gunicorn, dev uses runserver). Dockerfile,
.env.example,
.gitignore (include .env and media/), ruff config, pytest config with pytest-
django.
A README with the 3 commands to run it locally. Do not add models yet.

```

Step 2 — models, migrations, admin, seed

Implement every model in the Data model section of docs/PROJECT_SPEC.md exactly:

Office, Department, Village, Category, custom User (AbstractUser + office, department, phone),

Complainant, Complaint, Referral, InquiryReport, PortalSubmission, Document, Event, Tag.

Use TextChoices for every enum. Add all listed constraints (including the partial unique indexes) and indexes, plus a GIN SearchVector index and pg_trgm trigram indexes (enable the extension in a migration). Register master data and users in Django admin with list filters and search. Event must be read-only in admin. Create roles as Django groups in a data migration: Clerk, DepartmentOfficer, Reviewer, Admin, Auditor.

Add a management command `seed` that creates 1 office, the 9 departments from the spec, 7 villages (Bhattu Kalan, Tohana, Bhuna, Ratia, Jakhal, Bhardana, Mandi Dabwali), 5 categories, one user per role, and 30 fake complaints spread across every state by calling `apply_transition` (write a stub now, real one in step 3). Use fake data only.

Step 3 — state machine and tests

Implement `complaints/workflow.py` following the States and Transitions in `.github/copilot-instructions.md`. Public API:

- `apply_transition(complaint, action, actor, **data) -> Event`
- Raise `InvalidTransition` for any illegal (state, action) pair. Use `select_for_update` on the `complaint` and `transaction.atomic`. Each action validates its required data (department, document, note, reason, next_step). Write the Event payload with the ids of rows created.
- Also implement `complaints/services/pendency.py` (days at current office, total age, sla_days, ageing bucket) and `services/summary.py` (one plain-English summary sentence built from Events, no LLM).
- Tests (pytest, table-driven): every legal transition, every illegal one, round_no and attempt_no numbering, the one-open-referral rule, a full scenario (3 notice rounds, 2 portal rejections, close), reopen, pendency maths around dates, summary text.

Step 4 — list and detail screens

Build the main UI as Django templates + HTMX, matching the layout of the DM's prototype

`SDM_Complaint_App-3.html`: header bar, KPI strip, left 60% list with search box, filters (department, village, platform, status, category, date range, tag) and ageing chips, right 40% detail pane loaded with `hx-get`. On screens under 960px the detail opens full screen. Use one shared filter/search/pagination helper (`complaints/services/search.py`) with Postgres full-text + trigram search. Detail pane: header, status box with summary sentence, description, documents grouped by kind (served via a permission-checked view), timeline from Events, and only the action buttons valid for the current state. Each action opens a small HTMX form that calls `apply_transition` and re-renders the pane. All querysets go through `visible_complaints(user)`. Keep CSS light (Pico.css or Tailwind CDN).

Step 5 — intake

Build the New Complaint screen. Step 1: upload a PDF/JPG/PNG (max 10 MB). Save it as a Document (kind=complaint) keyed by SHA-256. Extract text with pdftotext; if no text layer, run Tesseract with hin+eng. Step 2: show a form pre-filled by simple regex guesses (CM Window grievance id, phone, name, village matched against the Village table, subject). Nothing is saved as a Complaint until the user clicks Save. Before save, run a duplicate check (same external_ref, same phone, or trigram-similar subject in the same village) and show possible matches with a 'mark as related' option. On save: create Complainant (reuse by phone), Complaint (ref_no auto C-YYYY-NNNN per office), link the Document, write COMPLAINT_REGISTERED. Add poppler-utils and tesseract-ocr (+ hin) to the Dockerfile. Tests for ref_no generation, duplicate detection and the dedupe-by-hash rule.

Step 6 — permissions and the overdue page

Implement complaints/permissions.py with visible_complaints(user) and can(user, action, complaint) for the roles in docs/PROJECT_SPEC.md (Roles, access and security). Department officers see and act only on complaints whose open Referral is with their department. Auditor is read-only. Enforce in views and in apply_transition callers. Build the Overdue Today page: open complaints past SLA grouped by department, oldest first, plus portal rejections awaiting action, with a print stylesheet. Tests for every role.

Step 7 — production readiness

Go through the Security checklist in docs/PROJECT_SPEC.md and make the code satisfy it:
 prod settings (DEBUG False, ALLOWED_HOSTS, secure cookies, HSTS), admin moved off /admin/,
 upload type/size validation, a Caddyfile, a prod docker-compose override with a caddy
 service, scripts/backup.sh (pg_dump + media tarball, keep 14 days) and scripts/restore.sh,
 and a README section on backup and restore. Do not choose a hosting provider.

Review checklist after each Copilot step

- ☐ Did it add any library or service not in the stack? Remove it.
- ☐ Does any code set `current_state` outside `workflow.py`?
- ☐ Do the tests pass, and do they test the rule, not just that the page loads?
- ☐ Are there migrations for every model change?
- ☐ Any secret, real complaint data or `.env` in the commit?

Open questions and data to collect

Coding can start on seed data; these answers are needed before the pilot goes live. Ask the DM and the dealing clerk.

Questions for the DM

- ☐ Do complaints from Jan Samvad, Samadhan Shivir, social media and walk-ins also go to the CM portal, or only CM Window ones? (Decides how often `close_without_portal` is used.)
- ☐ Can the office download complaints as PDFs from CM Window instead of printing and scanning? (If yes, intake needs no OCR.)
- ☐ Is the district on NIC eOffice / eFile? If yes, this tracker must sit beside it, not duplicate it.
- ☐ What SLA (days) does each department get for an inquiry report? 7? 15?
- ☐ When the portal rejects, what are the usual reasons, and who decides between resubmit and fresh inquiry?
- ☐ Should department officers log in themselves, or will only the SDM office staff use it?
- ☐ How long must complaint data be kept, and who may see complainant phone numbers?

- ☐ Where may the data be hosted for the pilot (commercial Indian cloud acceptable, or NIC only)? Who owns the hosting account?
- ☐ Confirm the understanding: this is an internal working tracker, not a parallel official record.
- ☐ What does the SDM want to see first thing each morning?

Data to collect

- ☐ Final list of departments / branches, each with head name, email, phone
- ☐ All villages and wards in the sub-division (with Hindi names if possible)
- ☐ Grievance categories the office uses
- ☐ SLA days per department or category
- ☐ Users: name, role, department
- ☐ 5–10 real complaint PDFs (anonymised) for testing intake
- ☐ Sample notice, inquiry report and ATR formats
- ☐ Complaint number format the office already uses, if any

Existing products reviewed

Nothing off the shelf models the notice → report → review → portal → rejection loop, so we build the case model ourselves and borrow ideas.

Product	What it is	Verdict
CM Window, Haryana	The state's official grievance portal	Upstream system, not a competitor. Our tracker is the office-side working layer; no public API, so uploads stay manual.
CPGRAMS	Central government grievance portal	Same: upstream, not a replacement
NIC eOffice / eFile	Government file-movement tracking	Check first whether the district uses it; if so, sit beside it

Product	What it is	Verdict
DIGIT PGR (eGov Foundation)	Open-source grievance module used by urban local bodies	Too heavy to run (Java microservices), but its data model is a useful cross-check
Frappe Framework / Helpdesk	Python low-code platform with roles, workflows, attachments	Fastest alternative if we did not want to write the app; rejected in favour of plain Django for control
Paperless-ngx	Open-source document store with OCR and search	Covers the document half only
Zammad / OTRS / FreeScout	Ticketing with SLA	Poor fit for multi-round, document-heavy loops
Camunda / Temporal	Workflow engines	Overkill; a small state machine is enough
NocoDB / Baserow / Appsmith	Better spreadsheets / low-code	A stop-gap, not a product
ServiceNow, Salesforce Public Sector, Zoho	Commercial case management	Out of budget and procurement reach for a sub-division

Decision log

Newest first. Add a row whenever a decision changes.

Date	Decision	Why
3 Oct 2026	Added state <code>APPROVED</code> (satisfied, not yet uploaded) and actions <code>close_without_portal</code> , <code>portal_rejected</code> with <code>next_step</code>	Found while writing this spec: "satisfied" and "uploaded" happen on different days, and some sources may not need a portal upload
3 Oct 2026	Start coding now; deployment decided later	Hosting blocked; code does not depend on host
3 Oct 2026	Use Copilot with a repo instructions file + step prompts	Keep the spec in the repo as the source of truth

Date	Decision	Why
28 Sep 2026	Vercel rejected	Serverless, no disk, no Postgres; wrong shape for Django
25 Sep 2026	NIC MeghRaj (cloud.gov.in) not for the pilot	Govt-only onboarding through the department
24 Sep 2026	Oracle Always Free tried in Mumbai; ARM shape out of capacity	Fallbacks: E2.1.Micro, paid Indian VM, Render/Railway
24 Sep 2026	Free domain via DuckDNS; HTTPS via Caddy	Pilot must be free
24 Sep 2026	Notifications on hold	v1 is only a tracker
23 Sep 2026	Django + Postgres + HTMX + local disk + cron on one server; cut Redis, Celery, S3, React, search engine, managed DB	No over-engineering at ~6,000 complaints a year
23 Sep 2026	Complaints are one typed page: <code>pdftotext</code> first, Tesseract for scans, no LLM	Removes OCR cost and complexity
23 Sep 2026	Append-only event log + derived state; loop stored as rows (Referral, InquiryReport, PortalSubmission)	The core fix for the to-and-fro clutter
23 Sep 2026	Replace Google Sheets + Drive with a real database	No constraints, transactions, access control or audit in Sheets
23 Sep 2026	<code>office_id</code> on main tables	Keeps multi-office rollout possible for free